Project Title: AI-Powered Smart Banquet & Event Planning Platform

System Requirement Specification (SRS)

Programme: Bachelor of Technology in Computer Science & Engineering (2026)

Institution: School of Engineering, Babu Banarasi Das University, Lucknow – Department of Computer Science & Engineering

Project Supervisor: Prof. Adesh Kumar Srivastava, Associate Professor, Department of Computer Science & Engineering

Document Status: Requirement baseline for the current semester. Implementation is planned for the Major Project semester.

Submitted by

Team Member	University Roll No.
Divya Mishra	1230432278
Divyanshu Rawat	1230432280
Deepali	1230432260
Devansh Kumar Rai	1230432266

Target Region: Supported cities and areas are limited to a controlled location list in the prepared dataset (Lucknow is used as an example in this document). Currency: a single currency per requirement profile (INR in the illustrative examples).

Target Platforms:

- **Customer:** Streamlit-based customer interface (customer view)
- **Vendor:** Streamlit-based vendor interface (vendor dashboard)
- **Admin:** Administrator functions are defined in Sections 2.3 and 3.3

1. Executive Summary & Business Objectives

AI-Powered Smart Banquet & Event Planning Platform is proposed as an AI-assisted, data-driven decision-support system for event planning. A customer describes an event through a structured form or in natural language; the platform extracts a structured requirement profile, filters suitable venues and vendors, ranks them according to customer preferences, explains the recommendation, supports comparison, builds a budget-aware plan, recalculates the plan when requirements change, and records a protected booking confirmation. Vendors maintain their own prices, packages, services and date-wise availability.

Conventional banquet and event websites act mainly as listings: a customer must search manually, compare venues and services without guidance, estimate the total cost separately, and restart the process whenever the guest count, budget or date changes. The proposed platform instead treats event planning as a connected decision-making workflow and is therefore **not** a simple banquet booking or search-and-listing website.

Background & Significance

Event planning for occasions such as weddings, engagements, birthdays, receptions and corporate functions involves several interconnected decisions: selecting a suitable venue, choosing vendors and venues for catering, decoration, photography and other services, and managing the overall budget. These decisions become harder as the number of guests grows, as requirements change during planning, or when the user has distinct priorities for different services. Vendor and venue information (prices, packages and availability) changes frequently and is not always reflected accurately across the sources a user consults, which creates a need for a platform where vendors update this information themselves and where a plan can be adjusted whenever requirements change.

Traditional planning requires a user to gather and compare information from multiple independent sources. Venue capacity, price, food options, decoration quality and photography services are evaluated separately, and the user must combine them into a coherent plan that fits a defined budget. The significance of this project lies in three aspects:

- **Personalization:** two users with an identical budget and guest count may need different recommendations (one may prioritize decoration, another food or photography); a preference-aware scoring approach represents these priorities and ranks options accordingly.
- **Reliability of information:** because prices, packages and availability are maintained directly by vendors rather than by a central administrator, the platform is expected to reduce the likelihood that a user plans around outdated or unavailable options.
- **Adaptability:** because requirements can change during planning, the platform recalculates the recommended plan and budget rather than requiring the process to be repeated manually.

Problem Statement

The problem addressed is the absence of an integrated platform that lets a user select a suitable combination of vendors and venues while receiving a preference-aware recommendation, an optimized cost estimate, and dynamically updated vendor and venue information, including current prices and availability maintained directly by the vendor. A related problem is that requirements such as guest count, budget or event type can change during planning, and existing systems generally do not recalculate the plan automatically when this happens.

Existing event and vendor discovery systems present vendors and venues independently rather than as part of a complete event plan, and the information shown is often maintained by a central administrator, which increases the likelihood of outdated prices or availability. The user must manually compare vendors and venues across capacity, price, rating and service quality, calculate the complete event budget separately, and repeat this work whenever requirements change.

Gaps in Existing Systems

- Vendor and venue information is handled and presented independently, rather than as part of an integrated event plan.
- Recommendations do not adequately account for individual service priorities.
- Users must manually compare parameters such as capacity, price, rating, food quality, decoration and photography across multiple sources.
- Budget calculation is typically separated from vendor and venue recommendation.
- Price and availability are often maintained by a central administrator rather than updated directly by the vendor, which increases the likelihood of outdated listings.
- Existing systems generally do not recalculate the event plan when a customer’s requirements change.
- A confirmed booking has no standard mechanism to protect its agreed price and details from later changes to a vendor’s listing.

Need for the Proposed Work

The proposed system combines requirement understanding, recommendation, comparison, cost optimization and plan recalculation with vendor-managed data within a single platform. It converts user requirements into structured parameters, uses them to rank available vendors and venues, and generates a budget-optimized event plan that can be recalculated whenever requirements change. Vendors maintain their own prices, packages and availability directly, which keeps the information used for recommendation current. The system is intended to support the user's decision-making rather than to replace it or to process real-world financial transactions.

Development Problem

The development problem is to design a recommendation and cost-optimization pipeline that operates on structured vendor and venue data, incorporates user preferences into a ranking mechanism, produces explainable recommendations, and recalculates the plan when requirements change. A further requirement is a vendor-side data management workflow that keeps vendor and venue information current, together with a booking mechanism that protects the details of a confirmed booking from any subsequent change to a vendor's listing.

Key Business Goals

- **Requirement Understanding:** Capture event requirements through a structured form and natural-language input, convert them into a structured requirement profile, and ask for clarification when needed.
- **Intelligent Recommendation:** Filter venues and vendors using hard constraints, rank feasible options using customer preferences and attributes, explain why an option is recommended, and allow comparison of alternatives.
- **Budget-Aware Planning & Recalculation:** Generate a budget-aware plan, optimize the combination of venue, services and packages, and recalculate the plan when requirements change, without a full restart.
- **Streamlined Vendor Operations:** Let vendors manage prices, packages, services, availability and offline bookings through a vendor dashboard.
- **Protected Booking Records:** Store confirmed bookings with the agreed price and details, protected from later listing changes, and generate a digital booking confirmation record.
- **AI Integration:** Demonstrate the integration of data processing, NLP-based requirement extraction, preference-aware recommendation, budget optimization, vendor-managed data and protected booking records within a practical B.Tech CSE software prototype.

Table 1.1: Project objectives and the capability that satisfies each objective

ID	Objective	Primary Capability
O1	Capture event requirements through a structured form and natural-language input.	Customer module, NLP module
O2	Convert user input into a structured requirement profile and ask for clarification when needed.	Requirement understanding
O3	Filter venues and vendors using hard constraints.	Recommendation module
O4	Rank feasible options using customer preferences and attributes.	Recommendation module
O5	Explain why an option is recommended and allow comparison of alternatives.	Explainability and comparison
O6	Generate a budget-aware plan and optimize the combination of venue, services and packages.	Budget module
O7	Recalculate the plan when requirements change, without a full restart.	Recalculation module

ID	Objective	Primary Capability
O8	Let vendors manage prices, packages, services, availability and offline bookings.	Vendor dashboard
O9	Store confirmed bookings with the agreed price and details, protected from later listing changes.	Booking module, database
O10	Generate a digital booking confirmation record.	Booking module

Table 1.2: Key functional goals

Goal	Expected Function
Personalization	Rank vendors and venues according to the user's stated priorities.
Budget optimization	Recommend a combination of vendor and venue services that fits the stated budget.
Explainability	Present the factors that support each recommendation.
Adaptability	Recalculate the plan and budget when the customer's requirements change.
Vendor-managed data	Allow vendors to maintain current prices, packages and availability.
Booking protection	Preserve the agreed price and details of a confirmed booking.

System Scope

The scope covers customer registration and login, event requirement collection, venue and vendor discovery, hard-constraint filtering, preference-aware ranking, explainable recommendation, comparison, cost estimation, budget optimization, event plan recalculation, vendor-side data management, booking confirmation and structured data storage.

Table 1.3: Scope areas and required capability

Scope Area	Required System Capability
Customer application	Requirement entry, recommendations, comparison, budget planning, recalculation and booking confirmation.
NLP and AI	Natural-language requirement extraction, preference-aware ranking and explanation support.
Budget planning	Component-wise cost estimation, budget breakdown and constrained plan selection.
Vendor management	Prices, packages, services and date-wise availability updates.
Booking	Confirmed booking record with protected agreed price and details.
Database	MySQL storage for users, vendors, venues, services, packages, pricing, availability, requirements, recommendations and bookings.

Project Timeline

Table 1.4: Academic timeline of project activities

Phase	Activities
Current semester	Problem identification, requirement gathering and analysis, literature review, existing-system study, dataset identification and preparation, architecture planning, database planning, AI/ML approach planning, methodology, documentation and synopsis finalization.

Phase	Activities
Next (final) semester	Implementation of the database, AI/NLP components, recommendation module, budget optimization, vendor dashboard and booking module; integration; testing; Final Major Project submission.

This document records only planned implementation. It does not claim that any implementation module has been completed.

Table 1.5: Six-week work plan for the current semester

Week	Planned Work	Expected Deliverable
1	Problem identification and requirement analysis.	Finalized problem statement and objectives.
2	Literature review and study of existing systems.	Completed literature review.
3	System architecture and module planning.	Proposed architecture and module list.
4	Dataset identification, preparation and database planning.	Identified data sources and draft database schema.
5	Selection of AI/ML approaches and methodology preparation.	Finalized methodology and algorithms.
6	Documentation, review and synopsis finalization.	Completed and submitted synopsis.

Assumptions & Constraints

- The prototype operates on a curated dataset of venues, vendors, services and packages prepared during the project.
- Prices produced by the system are estimates derived from that dataset and are not negotiated market prices.
- Real-world payment processing is not part of the prototype.
- The system is developed by a four-member student team using open-source technologies.
- The booking confirmation records an agreed amount without processing an actual financial transaction.
- The accuracy of recommendations and cost estimates depends on the quality and coverage of the dataset used.
- A dispute concerning an already confirmed booking is resolved directly between the customer and the vendor and is outside the scope of the platform.
- Recommendations are decision support and should be verified by the user before a final commitment is made.

Requirement Conventions

- **“shall”** – mandatory for the prototype; **“should”** – desirable; **“may”** – optional, subject to implementation decisions.
- Items labelled **FUTURE SCOPE** are outside the prototype commitment.
- Each major function defines its expected input, processing responsibility, output, validation behaviour and failure condition so that it remains measurable and testable.

Expected Outcomes

The expected outcome is a working prototype that demonstrates how preference-aware recommendation, budget optimization, plan recalculation and vendor-managed data can support event-planning decisions. These are planned deliverables of the following semester’s implementation phase.

- A user-friendly interface for entering event requirements through a structured form or natural language.
- Personalized vendor and venue recommendations based on budget, capacity, location and preferences.
- Vendor recommendations for services such as catering, decoration and photography, aligned with user priorities.
- A clear explanation of the leading factors behind each recommendation.
- An approximate, budget-optimized event cost with a component-wise breakdown.
- Automatic recalculation of the recommended plan and budget when requirements change.
- A vendor dashboard through which vendors independently maintain prices, packages, services and availability.
- A protected, fixed record of the agreed price and details for each confirmed booking, with a digital booking confirmation record accessible to the customer and the vendor.
- Structured storage of vendor, venue, pricing, availability and booking data in MySQL.

Advantages

- Personalized recommendations in place of generic, unranked listings.
- Integrated recommendation, comparison, budget optimization and plan recalculation within a single workflow.
- Vendor-managed pricing and availability, which keeps the underlying dataset current.
- A protected booking record that keeps a confirmed booking unaffected by later vendor-side changes.
- A modular architecture that can be extended with additional service categories.

Limitations

- The accuracy of recommendations and cost estimates depends on the quality and coverage of the dataset used.
- Prototype prices may differ from real, negotiated prices at the time of an actual booking.
- Availability reflects the updates maintained by vendors through the dashboard rather than an external, real-time source.
- The performance of more advanced machine-learning models depends on the size and quality of the dataset available at implementation time.

Innovation Potential

The strongest innovation opportunity lies in combining preference-aware recommendation and explanation, budget optimization, automatic plan recalculation and a vendor-managed two-sided data model within a single platform. Rather than functioning as a static listing service, the platform maintains current information through direct vendor participation, adapts the plan when requirements change, and protects the integrity of a confirmed booking, moving from a simple search interface toward an integrated planning and booking platform.

Literature Review & Research Gap

The platform draws on recommender systems, machine learning, natural language processing, optimization and two-sided marketplace design.

- **Recommender systems:** commonly built with collaborative filtering, content-based or hybrid approaches (Ricci, Rokach and Shapira, 2022). Collaborative filtering needs interaction data from many users, which is difficult for a student prototype; a content-based, attribute-based approach

using structured features (price, capacity, location, service ratings) with similarity or weighted scoring (Resnick and Varian, 1997) is adopted.

- **Machine learning for ranking and optimization:** item and user attributes are converted into a relevance score after feature scaling and categorical encoding (Pedregosa et al., 2011). Selecting a complete vendor-and-venue combination within a budget is a constrained combination-selection problem related to knapsack-style allocation (Geron, 2022). The platform applies scoring-based ranking for recommendation and a constrained selection approach for budget optimization.
- **NLP for requirement extraction:** rule-based pattern matching and named-entity recognition extract slots such as guest count, budget, location and event type from short statements; at this project scale a rule-based or library-based approach is more realistic than training a language model.
- **Data-driven cost estimation:** transparent, component-level formulas combining guest-dependent and package-dependent costs are the primary method, with regression-based prediction as a possible extension as historical data grows.
- **Vendor-managed marketplaces:** letting suppliers maintain their own listings keeps the dataset current and reduces administrative effort; this principle informs the vendor dashboard.
- **Research gap:** these techniques are typically applied in isolation, and existing banquet and event-planning platforms rarely combine NLP requirement extraction, preference-aware recommendation with explanation, budget optimization, plan recalculation and vendor-managed pricing and availability in one connected workflow.

References

- F. Ricci, L. Rokach and B. Shapira, Recommender Systems Handbook, 3rd ed., Springer, 2022.
- P. Resnick and H. R. Varian, “Recommender Systems,” Communications of the ACM, vol. 40, no. 3, pp. 56–58, 1997.
- F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- W. McKinney, Python for Data Analysis, 3rd ed., O’Reilly Media, 2022.
- A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed., O’Reilly Media, 2022.
- Python Software Foundation, Python Documentation.
- Streamlit Documentation, Streamlit Official Documentation.
- MySQL Documentation, MySQL Reference Manual.

2. User Roles, Onboarding & Access Control

The platform distinguishes permissions according to role. The roles identified for the prototype are Customer, Vendor and Administrator. Each role accesses only the functions required for its responsibilities.

2.1 Customer Role

Onboarding & Auth:

- Register and log in with the minimal information necessary for planning and booking.
- Email + Password: valid email format, unique among accounts; password stored as a salted hash; weak or missing passwords are rejected.
- Each event requirement profile is associated with the owning customer account.

Profile Management:

- Own profile, event requirements, recommendations, comparisons, budget plans, recalculations, bookings and confirmations.
- Saved payment methods: not applicable to the prototype (online payment is FUTURE SCOPE).
- **Not permitted:** modifying vendor listings or other customers' records.

2.2 Vendor Role

Onboarding & Auth:

- Registration with the role Vendor, capturing vendor identity and the venue or service information needed to maintain listings.
- Business name and service category are mandatory; vendor onboarding is blocked until they are complete.
- A vendor is linked to every venue, service, package and availability record that it manages.

Access:

- Own profile, venue and service records, prices, packages, availability, offline-booking updates and bookings made against own listings.
- **Not permitted:** modifying other vendors' records or customers' requirement data.

2.3 Administrator Role

Role Types:

- Administrator. Administrator accounts are created separately and cannot be self-registered.

Access Control:

- Authentication and authorization are enforced in the application logic and the database access layer; protected operations do not rely only on hiding interface controls.
- Access-sensitive operations (updating vendor data, confirming a booking, modifying administrative records) are associated with the authenticated account and the ownership of the record concerned.

Capabilities:

- Record review, data-quality oversight, user and vendor account management, dataset import review and controlled administration.
- **Not permitted:** silently altering confirmed booking values or replacing vendor responsibility for current prices.

Table 2.1: Roles and access boundaries

Role	Permitted Access	Not Permitted
Customer	Own profile, event requirements, recommendations, comparisons, budget plans, recalculations, bookings and confirmations.	Modifying vendor listings or other customers' records.
Vendor	Own profile, venue and service records, prices, packages, availability, offline-booking updates and bookings made against own listings.	Modifying other vendors' records or customers' requirement data.
Administrator	Record review, data-quality oversight, user and vendor account management, controlled administration.	Silently altering confirmed booking values or replacing vendor responsibility for current prices.

Table 2.2: Input validation during registration

Input	Validation Rule	Failure Behaviour
Name	Non-empty text	Display field-level message
Email	Valid format; unique among accounts	Reject; ask for a different email
Password	Minimum length defined by the system; stored as a hash	Reject weak or missing password
Role	One of Customer or Vendor (Administrator created separately)	Reject unknown role
Vendor details	Business name and service category required for vendors	Block vendor onboarding until complete

The system shall provide clear validation messages when required registration or profile information is missing or invalid.

3. Detailed Feature Specifications by Platform

Table 3.1: End-to-end planning workflow

Step	Workflow Stage	System Responsibility	Module
1	Requirement entry	Accept event type, location, guest count, budget and service preferences from the customer as a form or natural-language input	Customer, NLP
2	Requirement understanding	Extract parameters, validate, request clarification if required	NLP
3	Data retrieval	Retrieve current vendor and venue records from MySQL	Database
4	Hard filtering	Remove options failing capacity, location, date or event-type constraints	Recommendation
5	Ranking	Score and order feasible options using customer preferences	Recommendation
6	Explanation and comparison	Show reasons for ranking; compare selected alternatives	Recommendation, UI
7	Budget planning	Estimate component costs and optimize within the budget	Budget
8	Requirement change / recalculation	Customer updates guest count, budget, event type or another major requirement; recommendation and budget modules are re-run to produce an updated plan	Recalculation
9	Vendor update	Vendor modifies price, package, service or availability through the vendor dashboard; changes feed all later requests	Vendor dashboard, Database
10	Confirmation and output	Store agreed price and details as a protected booking record and generate the digital confirmation; output is the recommended plan, estimated budget and booking confirmation record	Booking, Database

3.1 Customer Application (Streamlit Interface)

1. Requirement Entry & Understanding

- **Input Modes:** A structured form (event type, date, location, guest count, budget, service preferences, optional services, priorities) or a natural-language event description. Both are converted into a common structured requirement profile that is the single input to the recommendation and budget modules.
- **Validation:** Guest count must be numeric and positive, the budget a valid non-negative amount, the date acceptable, and all mandatory planning fields present before recommendation begins.
- **Confirmation of Extracted Values:** Values extracted from natural language are shown as an editable structured summary so the customer can confirm or correct them before processing continues.
- **Clarification:** If an important value is missing, ambiguous or extracted with low reliability, the system requests clarification and never silently assumes an incorrect value.
- **Failure Behaviour:** For empty, contradictory (e.g., two different guest counts) or unrecognisable input, the system displays a specific message identifying what is missing and retains the data already entered.
- **Versioning:** Each saved profile is stored with its owner, creation time and last-modified time so earlier versions can be referred to during recalculation and testing.

Table 3.2: Event requirement profile

Field	Type	Validation	Mandatory
Event type	Category	Must belong to the supported event-type list (for example wedding, engagement, birthday, reception, corporate function)	Yes
Event date	Date	Valid calendar date, not in the past	Yes
Location	Text/category	Must match a supported city or area in the dataset	Yes
Guest count	Integer	Positive whole number	Yes
Total budget	Decimal	Non-negative amount in a single currency	Yes
Service preferences	Set	Chosen from supported service categories such as catering, decoration, photography, DJ/entertainment	No
Optional services	Set	Services that may be dropped or added depending on cost	No
Priorities	Ordered set/weights	Relative importance of venue, price, rating or individual services	No

Table 3.3: Input, processing and output for requirement capture

Input	Processing	Output
Form fields	Type, range and completeness validation	Validated requirement profile
Free-text event description	NLP extraction, normalisation and validation	Draft profile with confidence/missing flags
Clarification answer	Merge answer into the draft profile and re-validate	Completed requirement profile
Customer edit	Overwrite the field and mark the profile as changed	Updated profile, triggers recalculation

Table 3.4: Illustrative conversion of natural-language input

Customer Statement (example)	Structured Result
“Wedding for around 300 guests in Lucknow with a budget of 5 lakh”	event_type = wedding; guest_count = 300; location = Lucknow; budget = 500000
“We want good photography and a DJ”	services = {photography, DJ}
“Venue matters most to us”	priority: venue = high
“A party for my friends next month”	event_type = party (to be confirmed); date, guest count and budget flagged as missing

2. Discovery, Recommendation, Comparison & Explainability

- **Hybrid Approach:** The recommendation module combines hard-constraint filtering with preference-aware ranking; not every available listing is treated as equally suitable.
- **Exclusion Reasons:** Options failing any hard constraint are excluded from ranking, and the reason for exclusion is retained so it can be reported when no option remains.
- **Relevance Scoring:** Remaining options are scored using attribute-based (content-based) methods covering price fit against budget, capacity fit, location match, rating or evaluation attributes, available services and package contents, and the customer’s stated priorities.
- **Reproducibility:** The scoring approach, attribute weights and normalisation method are documented so the ranking can be interpreted and reproduced; ties are resolved by a deterministic tie-breaking rule.
- **Explainability:** Each leading option shows the main factors behind its rank (e.g., capacity and location satisfied, price within budget, venue given high priority). Explanations are generated from the actual scoring factors and are never fabricated.
- **Comparison:** Customers can select two or more options and view them side by side: price, capacity, rating or evaluation attributes, services included, package contents, availability on the requested date, and recommendation score with its main factors.
- **Result Display:** A ranked list with a defined maximum number of results per page; the customer can select an option for budget planning.
- **Failure Behaviour:** When no option satisfies the hard constraints, the system reports which constraint eliminated the most candidates and suggests adjustments such as a different date or a smaller guest count. It never returns an empty screen without explanation.

Table 3.5: Hard constraints applied before ranking

Constraint	Rule	Data Used
Venue capacity	Requested guest count lies within the venue’s supported capacity	Venue capacity
Location compatibility	Venue or vendor serves the requested location or area	Location fields
Date availability	Venue or vendor is available on the requested date	Availability records
Event-type compatibility	Venue or vendor supports the requested event type	Event categories
Budget feasibility	Minimum possible cost of the option does not exceed the budget	Pricing data

Table 3.6: Recommendation pipeline

Stage	Requirement
Constraint filtering	Remove records that violate essential requirements.
Feature evaluation	Evaluate price, capacity, location, services, rating and preference attributes for each remaining option.
Preference scoring	Give greater weight to attributes the customer prioritises.
Ranking	Order suitable options by calculated relevance, with a deterministic tie-breaking rule.
Explanation	Display the major factors supporting each leading recommendation.
Comparison	Allow side-by-side evaluation of selected alternatives.

3. Budget Planning & Cost Estimation

- **Component-Based Costing:** Fixed components (e.g., venue rent, fixed-price service) and guest-dependent components (e.g., per-guest catering) are calculated separately so the customer can see how the total is formed.
- **Optimization:** The customer's budget is a hard constraint. The module searches feasible combinations of venue, services and packages at the scale of the prototype dataset, using an exhaustive, greedy or other documented selection method, and returns the combination with the best preference score whose total cost does not exceed the budget.
- **Priority Awareness:** A lower-cost option does not automatically replace a high-priority service if a feasible alternative satisfies both the budget and the priority.
- **Over-Budget Handling:** If the initial plan exceeds the budget, alternative plans are offered (e.g., a cheaper package, removal of an optional service, a different venue), each stating its cost saving and preference trade-off.
- **Output:** Total estimated cost, remaining or exceeded budget amount, and a component-wise breakdown of the selected venue, vendors, packages and optional services. A chart (bar or pie, using Matplotlib or Plotly) may be provided.
- **Disclaimer:** Figures are estimates based on the available dataset and may differ from real negotiated prices.
- **Failure Behaviour:** When no combination satisfies the budget, the system reports the minimum achievable cost and indicates which requirement (e.g., guest count or service selection) contributes most to the gap.

Table 3.7: Cost components

Component	Basis	Source
Venue cost	Fixed rate or per-guest rate	Venue pricing
Service cost	Fixed or per-guest, by service	Service pricing
Package cost	Package price replacing the individual service prices of its included items	Package records
Optional services	Added only if selected or if budget allows	Service pricing
Total estimated cost	Sum of selected components	Budget module

Table 3.8: Budget module input, processing and output

Input	Processing	Output
Budget limit	Set the maximum cost constraint	Budget status (within, near or over budget)
Guest count	Apply guest-dependent prices	Estimated component costs
Selected services and packages	Add service and package costs	Service-wise breakdown
Customer priorities	Prefer high-priority attributes in selection	Priority-aware plan
Available alternatives	Search feasible combinations	Optimized or revised plan with alternatives

4. Event Plan Recalculation

- **Core Requirement:** When a customer changes an important planning parameter after receiving an initial plan, the system updates recommendations and budget without requiring a restart of the planning process.
- **Triggering Changes:** Guest count, budget, event type, event date, service preference, and other significant requirements such as priorities or location.
- **Processing:** The system determines which parts of the previous plan are affected and re-runs the relevant filtering, ranking and budget logic using current vendor data, including prices, packages and availability changed after the original plan was created.
- **Change Summary:** The output shows the previous and new selection, the previous and new total cost, and any option that became unavailable or infeasible. The previous plan version is preserved for comparison.
- **Failure Behaviour:** If no feasible plan exists, the system states that the constraints cannot be satisfied together, identifies the responsible constraint, and offers relaxation options such as a different date, a higher budget or fewer optional services.
- **Scope Limit:** Recalculation applies only to unconfirmed plans; a confirmed booking record is never altered by recalculation.

Table 3.9: Required behaviour for each type of change

Changed Requirement	Expected System Behaviour
Guest count	Re-check capacity of the selected venue; recalculate guest-dependent costs; replace the venue if capacity is violated; update budget.
Budget	Re-run optimization against the new limit; find lower-cost feasible combinations when the budget decreases, or offer upgrades when it increases.
Event type	Re-filter venues and services for compatibility; recalculate the plan.
Event date	Re-check date availability for every selected venue and vendor; replace unavailable items.
Service preference	Re-score or replace the affected vendors and packages; update costs.
Priorities	Re-rank options; update the selected combination if the ranking changes.

Table 3.10: Recalculation input, processing and output

Input	Processing	Output
Changed parameter(s) and current plan	Identify affected stages; re-validate; re-filter, re-rank and re-estimate	Revised plan
Current vendor data	Apply latest prices and availability	Updated costs and options
Previous plan	Compute differences against the revised plan	Change summary

5. Booking & Confirmation

- **Plan Review:** The customer confirms a selected plan after reviewing the venue and vendor combination, services, packages and estimated cost.
- **Re-Validation:** Before confirmation, availability is re-checked and the cost recomputed with current data; the customer is informed if price or availability changed since the plan was shown.
- **Protected Record:** At confirmation, a protected booking record is created containing the agreed price, selected venue and vendors, selected services and packages, event date, relevant requirements and booking status. It is independent of the vendor's live listing, so later vendor price or package changes never alter it.
- **Double-Booking Prevention:** Confirmation marks the corresponding dates as booked in the availability records within the same transaction, so the same date is not offered or confirmed twice.
- **Digital Confirmation:** A digital booking confirmation record shows booking ID, customer, venue and vendors, services and packages, event date, agreed price breakdown and status, accessible to the customer and relevant vendors according to their permissions. It also carries a unique booking confirmation identifier and the confirmation date, and is securely maintained by the system.
- **Exclusions:** Real-world payment processing is not required (FUTURE SCOPE). Cancellation and modification of confirmed bookings are not required in the prototype; if implemented later, they shall act on the booking record explicitly and retain a history of the original agreed values.

Table 3.11: Booking stages and required behaviour

Stage	Required Behaviour	Failure Behaviour
Plan review	Show selected items, estimated total and requirements	Block if the plan is incomplete
Re-validation	Check availability and current price	Notify the customer of changes
Confirmation	Store agreed price and details as a fixed record	Roll back if any step fails
Price protection	Retain confirmed price despite later listing changes	Reject any update to confirmed values
Availability update	Mark confirmed dates as booked	Reject double booking
Confirmation output	Generate retrievable digital confirmation	Report error and retain booking state

3.2 Vendor Dashboard (Streamlit Interface)

1. Dashboard & Operations Overview

- Shows the vendor its current listings, upcoming bookings made through the platform, and the availability calendar of each listing in a tabular or list form.
- Vendor-side summary views may use Matplotlib or Plotly charts.

2. Venue, Service & Package Management

- Structured records are maintained for venues and service vendors so that recommendation, comparison and cost estimation operate on consistent attributes.
- **Venue records:** venue name, location, capacity, supported event categories, pricing information, rating or evaluation attributes where available, and availability information.
- **Service-vendor records:** vendor identity, service category, service name, price, package details, service attributes and date-wise availability where applicable.
- Vendors can create, view, update and deactivate their own venue, service and package records. Permanent deletion is restricted so that records referenced by existing bookings are preserved.
- **Pricing Configuration:** Fixed or per-guest price basis for services and venues; package prices replace the individual prices of included services.
- Price, package, service and availability changes are stored immediately and used by every subsequent recommendation, cost calculation and recalculation request.
- **Data-management function, not AI:** Price, package and availability management is implemented as direct database update functionality rather than a machine-learning component; a vendor updates a record and the value is immediately reflected in the data used by the recommendation and budget optimization modules.
- Vendor-managed data is validated for missing values, invalid prices, inconsistent categories and duplicate records before use by the recommendation and budget modules.

Table 3.12: Data groups managed through the vendor dashboard

Data Group	Representative Fields
Vendor profile	Vendor ID, business name, contact details, service category, status
Venue	Venue ID, vendor ID, name, location, capacity range, supported event types, base price, per-guest price, rating attributes
Service	Service ID, vendor ID, category, service name, price basis (fixed or per guest), attributes
Package	Package ID, included services, package price, applicable event types, conditions
Availability	Date, venue or vendor ID, status (available, unavailable, booked), source of update (platform booking or offline booking), last updated

Table 3.13: Vendor operations, validation and failure behaviour

Operation	Validation	Failure Behaviour
Add or update price	Numeric, positive, within permitted precision	Reject and keep the previous value
Create package	Included services must exist and belong to the vendor	Reject with list of invalid items
Mark date unavailable	Valid date; not already part of a confirmed platform booking	Warn about conflict; require explicit resolution
Update capacity	Positive integer; minimum not greater than maximum	Reject inconsistent range
Deactivate listing	Listing must belong to the vendor	Reject unauthorised request

3. Availability & Offline Booking Management

- Vendors set date-wise availability and can mark a date unavailable because of an offline booking or other commitment.
- Such updates affect subsequent recommendations and cost calculations; the platform never recommends an unavailable venue or service for a conflicting customer request.
- On an offline booking, the vendor marks the date unavailable; the platform then excludes that date from the availability shown to other customers for that vendor or venue, reducing the likelihood of a scheduling conflict between an offline booking and a booking made through the platform.
- Marking a date already part of a confirmed platform booking as unavailable raises a conflict warning that requires explicit resolution.

4. Bookings

- Vendors view bookings made through the platform against their own listings and the related digital booking confirmation, according to their permissions.

3.3 Administrator Panel

1. Account, Record & Data-Quality Oversight

- Controlled management of users, vendors, datasets and system records.
- Dataset import review and oversight of data quality (missing values, duplicates, inconsistent categories, invalid numbers, capacity or price conflicts).

2. Dispute Handling & Auditability

- Administrators shall not alter the agreed values of confirmed bookings.
- Changes to prices, availability and booking status store the time of the change and the account that made it, so that disputes can be examined.
- A dispute concerning an already confirmed booking is resolved directly between the customer and the vendor and is not automatically resolved by the platform (outside platform scope).

3.4 AI, NLP & Recommendation Services (Cross-Platform)

- AI-assisted components are reproducible, testable modules separate from the user interface, so they can be unit-tested with fixed inputs and expected outputs.
- **Requirement Understanding:** Extracts at minimum event type, guest count, budget, location and service preferences from natural-language input and converts them into the structured profile (Table 3.2).
- **Baseline Method:** Rule-based pattern matching, keyword dictionaries and regular expressions. A pre-trained NLP library such as spaCy may be used for tokenisation, entity recognition and matching where it improves accuracy at prototype scale.
- **Field-Level Output:** For every field the extractor returns the extracted value, its source text span and a status of extracted, ambiguous or missing. Unsupported, ambiguous or conflicting expressions are sent for clarification and never converted into arbitrary values.
- **Recommendation Method:** Rule-based scoring, attribute-based methods using Scikit-learn utilities (feature scaling, similarity measures), or a hybrid, depending on dataset size. The chosen method and the reason for its selection are documented.
- **Budget Method:** Transparent, deterministic calculations. A predictive cost model may be considered only if sufficient historical pricing data becomes available; this is not committed for the prototype.

- **Hybrid Rationale:** The system combines deterministic business rules with similarity- or scoring-based ranking, because some requirements (e.g., guest capacity) are strict constraints while others (e.g., service priorities) are preferences that should influence ranking rather than eliminate options.
- **Algorithm A – Preference-Based Ranking:** Each candidate is assigned a normalized weighted score: **Score = wV(Venue) + wF(Food) + wD(Decoration) + wP(Photography) + wB(Budget Fit) + wL(Location)**. Weights follow the priorities stated by the user (e.g., if decoration is high priority and photography low, the decoration weight is increased and the photography weight reduced). The exact weight values are calibrated and validated during implementation.
- **Algorithm B – Budget Optimization:** Treated as a constrained combination-selection problem rather than a simple additive calculation. Given the ranked vendors and venues per service category, the module selects the combination that best satisfies the user's preferences while keeping the total estimated cost within, or close to, the stated budget: **Total Cost = Venue Cost + (Guests × Food/Plate Cost) + Decoration Cost + Photography Cost + Other Service Costs**. When the best-ranked combination exceeds the budget, lower-cost alternatives are proposed for one or more service categories and the total is recalculated. (Section 3.1 treats the budget as a hard constraint; the exact handling of "close to" the budget is to be finalized during design.)
- **Recalculation Logic:** Event Plan Recalculation reuses the same scoring and optimization methods (Algorithms A and B) on the updated requirement values.
- **Evaluation:** Components are evaluated for output correctness, consistency for repeated identical input, response time and behaviour on invalid input. If a trained model is used, its version, preprocessing steps, feature definitions, dataset version and evaluation results are documented, and random operations use fixed seeds.
- **Limitations:** Known limitations (unsupported phrasings, dependence on the prepared dataset) are documented in the final report.

Table 3.14: Extraction targets and handling of variations

Entity	Variations to be Handled	Handling
Event type	Synonyms such as marriage and wedding; plural and case differences	Map to the canonical event-type list
Guest count	"300 guests", "around 300", "300 people", "300 log"	Extract number; mark approximate expressions
Budget	"5 lakh", "5L", "Rs. 500000", "INR 5,00,000", "under 5 lakh"	Normalise to a numeric amount; flag upper-limit wording
Location	City or area names, spelling and case variations	Match against the known location list
Service preferences	"photographer", "photography", "DJ", "decor", "catering"	Map to canonical service categories
Priorities	"most important", "must have", "not important"	Convert to priority levels

Table 3.15: AI components and requirements

Component	Requirement
NLP extraction	Convert natural-language requirements into structured event parameters with field-level status.
Recommendation	Rank suitable options using constraints and customer preferences.
Explainability	Identify the major factors responsible for each ranking.
Budget optimization	Select or adjust a combination of services to satisfy budget constraints.

Component	Requirement
Recalculation	Re-run the affected logic after major requirement changes.

4. Non-Functional & Technical Requirements

4.1 System Architecture & Tech Stack

- **Application Logic:** Python, with Pandas and NumPy for dataset manipulation and numerical processing, and Scikit-learn for scoring or similarity components where required.
- **Interface:** Streamlit, with separate customer and vendor views within a modular application design.
- **Database & Storage:** MySQL via MySQL Connector or SQLAlchemy (decided in the implementation design).
- **NLP Layer:** Rule-based patterns combined with an NLP library such as spaCy, as appropriate to the dataset and scope.
- **Charts:** Matplotlib or Plotly may be used for budget breakdown and vendor-side summary views.
- **Modularity:** The customer interface, vendor interface, NLP processing, recommendation logic, optimization logic and database components are developed and tested independently before integration. Modules communicate through defined function interfaces and data structures (requirement profile, option list, scored option list, plan, booking record), so a change in one module does not require rewriting unrelated modules.
- **Configuration:** Database credentials and configuration values are kept in environment variables or a configuration file excluded from the repository, never hard-coded. A dependency list with version numbers is maintained.
- **External Services:** Not required for the prototype; if introduced later, they are isolated behind a defined interface.
- **Extensibility:** The design allows later extension (e.g., new service categories or FUTURE SCOPE items) without redesigning the core workflow.
- Cloud deployment is FUTURE SCOPE; the prototype runs in a local environment.

Table 4.1: Planned technology stack

Technology	Planned Use
Python	Application logic, data processing, NLP and recommendation implementation
Pandas / NumPy	Data cleaning, transformation, validation and numerical operations
Scikit-learn	Feature scaling, similarity measures and optional model components
spaCy / rule-based NLP	Natural-language requirement extraction
Streamlit	Customer and vendor interface
MySQL with MySQL Connector or SQLAlchemy	Operational database and persistence
Matplotlib / Plotly	Charts for budget breakdown and vendor views
Git / GitHub	Version control and repository management

Table 4.2: Module interfaces

Module	Receives	Returns
NLP / requirement module	Free text or form values	Requirement profile with field status
Recommendation module	Requirement profile, current data	Ranked options with explanation factors
Budget module	Requirement profile, selected options	Plan with cost breakdown and alternatives
Recalculation module	Changed profile, previous plan	Revised plan and change summary
Booking module	Confirmed plan	Booking record and confirmation

Conceptual flow: Customer Input (Form/NLP) → Requirement Extraction → Recommendation and Comparison Engine → Budget Optimization → Event Plan Recalculation (on requirement change) → MySQL (Vendor, Venue and Booking Data) ↔ Vendor Dashboard (Price and Availability Updates) → Booking Confirmation Module → Digital Booking Confirmation.

Table 4.3: Module description (to be implemented in the following semester)

Module	Description
Data Collection	Maintains vendor, venue, service and pricing records. The initial prototype dataset is created from structured sample data and, where permitted, publicly available information, with consistent fields to support recommendation and cost estimation.
Data Preprocessing	Pandas is used for cleaning, transformation and validation. Missing values, inconsistent text, duplicates and numerical formatting issues are handled before use; categorical attributes are encoded and numerical features normalized where required.
Event Requirement Understanding (NLP)	Converts form-based or natural-language input into event type, guest count, budget, location and service preferences using rule-based pattern matching and NLP techniques suited to the project scale.
Recommendation and Comparison	Combines hard filters (e.g., capacity, location) with preference-aware scoring to return a ranked list with an explanation of the leading factors, rather than a single unexplained result.
Budget Optimization	Estimates the cost of the recommended combination and proposes adjustments when the estimate exceeds the stated budget.
Event Plan Recalculation	Re-runs recommendation and budget optimization when a major requirement changes and presents an updated plan without restarting the process.
Vendor Dashboard	Separate interface for updating prices, packages, services and date-wise availability and marking dates unavailable after offline bookings. It performs standard data-management operations with no machine-learning component.
Booking and Confirmation	Records a confirmed booking as a fixed entry with the agreed price and details, independent of later listing changes, and generates the digital confirmation record.
Database	MySQL stores users, vendors, venues, services, pricing, availability, requirement profiles, and booking and confirmation records, and supports retrieval for the recommendation, budget and vendor dashboard modules.

Hardware & Software Requirements

Table 4.4: Hardware requirements

Component	Minimum / Recommended
Processor	Intel Core i3 or equivalent; i5 or equivalent recommended.
RAM	8 GB minimum; 16 GB recommended.
Storage	At least 10–20 GB of free project space.

Component	Minimum / Recommended
Display	Standard laptop or PC display.
Internet	Required for external libraries, package installation and any external NLP resources.

- **Operating System:** Windows 10/11 or equivalent.
- **Development:** Python 3.x; VS Code or an equivalent development environment; Git/GitHub for version control.
- **Database:** MySQL Server and MySQL Workbench; MySQL Connector or SQLAlchemy.
- **Libraries:** Pandas, NumPy, Scikit-learn, Streamlit, and an NLP library for requirement extraction.

4.2 Database & Data Management

- MySQL is the primary structured database. It stores users, vendors, venues, services, packages, availability, event requirements, recommendations, bookings and booking confirmations.
- Core entities have stable identifiers and defined relationships. No entities are included unless required by the project; additional tables are added during design only if a documented requirement needs them.
- **Data relationships:** A user can create multiple events; each event is associated with one requirement profile and one or more recommendation results. A venue is associated with a vendor profile and can offer multiple packages; a vendor can provide services across multiple venues or independently (e.g., catering, decoration, photography). Each confirmed booking is associated with exactly one event, one vendor-venue combination and one digital booking confirmation record. These relationships are represented using primary and foreign keys in the MySQL schema.
- Integrity is supported by primary keys, foreign keys, unique constraints, check constraints where supported, and transactions for operations that update more than one table.
- Booking records and their items store a snapshot of the agreed price and selected details, independent of live venue, service and package price fields.
- Indexes are created on the most-used filter columns: location, event type, capacity, date and vendor.
- Database operations are tested for insertion, retrieval, update, rejection of invalid data and relationship behaviour, including deactivation of a referenced record.
- Schema scripts and sample-data loading scripts are maintained under version control so the database can be recreated for testing.

Table 4.5: Core entities and representative attributes

Entity	Representative Attributes	Main Relationships
Users	User ID, name, email, password hash, role, created time	Owns requirements and bookings
Vendors	Vendor ID, user ID, business name, contact, category	Owns venues, services, packages
Venues	Venue ID, vendor ID, location, capacity, event types, prices, ratings	Has availability, appears in bookings
Services	Service ID, vendor ID, category, price, price basis	Included in packages and bookings
Packages	Package ID, vendor ID, price, applicable event types	Contains services

Entity	Representative Attributes	Main Relationships
Availability	Record ID, venue or vendor ID, date, status, source, updated time	Belongs to venue or vendor
Event requirements	Requirement ID, user ID, event type, date, location, guests, budget, priorities, version	Belongs to user; basis of recommendations
Recommendations	Run ID, requirement ID, option ID, score, rank, explanation factors	Generated from a requirement
Bookings	Booking ID, user ID, requirement ID, venue and vendor IDs, event date, agreed total price, status	Links user, vendor and requirement
Booking items	Booking ID, item type, item name, agreed price, quantity	Belongs to booking
Booking confirmations	Confirmation ID, booking ID, issue time, summary of agreed details	One per booking

4.3 Dataset Requirements & Data Governance

- The project prepares the structured data required for recommendation, comparison, cost estimation, availability management and booking. Dataset identification and preparation are part of the current semester; the data supports implementation in the following semester.
- No dataset statistics, survey findings or vendor facts are claimed in this document. Where data has not yet been collected, it is collected or prepared during the project and its source recorded.
- Initial data may be structured sample data and, where permitted, publicly available information. Demonstration or synthetic data is clearly labelled and never presented as verified real-world vendor information or availability.
- Every externally collected record carries provenance: the source, the collection date and a flag indicating real, sample or synthetic.
- Data cleaning identifies and handles missing values, duplicate records, inconsistent text values, invalid numeric formats, negative or zero prices, and category inconsistencies before records are used by any module.
- Categorical attributes are encoded and numerical features normalized where required before use by the recommendation module.
- A data dictionary defines each field’s name, data type, allowed values, whether mandatory, source or generation method, and the module that uses it.
- Dated snapshots of the prepared dataset are retained during development so evaluation results can be reproduced after the data changes.

Table 4.6: Data required by the project

Data Class	Required Content	Planned Source and Preparation
Venues	Name, location, capacity, supported event types, base price, rating attributes	Structured sample records; publicly available information where permitted; vendor-entered data
Vendors and services	Service category, service name, price basis, attributes	Structured sample records; vendor-entered data
Packages	Included services, price, applicable event types, conditions	Defined by the team from sample vendor offerings
Prices	Base price, per-guest price, package price, update date	Sample data with source and date recorded

Data Class	Required Content	Planned Source and Preparation
Ratings/evaluation attributes	Rating value or defined evaluation attributes	Sample or publicly available data; labelled as such
Availability	Date-wise status per venue or vendor	Generated demonstration data, later maintained by vendors
Event types and locations	Controlled lists of supported values	Defined by the team
Customer preferences	Priorities and service choices per requirement profile	Collected at run time through the application
Booking records	Agreed price, selected items, date, status	Created at run time on confirmation

Table 4.7: Data quality rules

Issue	Required Handling
Missing values	Mandatory fields block record acceptance; optional fields are stored as null and handled explicitly by the scoring logic.
Duplicate records	Detected using identifying fields (name, location, category) and resolved by merging or rejecting.
Inconsistent categories	Values are mapped to controlled lists of event types, locations and service categories.
Invalid numbers	Prices and capacities outside valid ranges are rejected or flagged for review.
Capacity or price conflicts	Minimum greater than maximum, or package price below the sum of mandatory components, is flagged.

Table 4.8: Data dictionary format (example entries)

Field	Type	Allowed Values / Rule	Used By
capacity_max	Integer	Positive; not less than capacity_min	Recommendation
event_type	Category	From the event-type list	NLP, Recommendation
price_basis	Category	Fixed or per_guest	Budget
availability_status	Category	Available, unavailable, booked	Recommendation, Booking
data_origin	Category	Real, sample, synthetic	Governance

4.4 Security, Privacy & Compliance

- Security controls are appropriate to a student software prototype. Protected customer, vendor and administrative functions require authentication and authorization.
- Users access only the records and functions permitted for their role: a customer cannot modify another customer’s requirement or booking, a vendor cannot modify another vendor’s records, and neither can modify administrative data.
- Passwords are never stored in plain text (secure salted hashing). Sessions are handled so protected pages cannot be accessed after logout.
- Database credentials and secrets are stored outside the public source code and not committed to Git.
- All customer forms, vendor updates and booking operations validate input; database queries are parameterised to reduce injection risk.

- **Data Privacy:** Only the personal information needed for planning and booking is collected.

Table 4.9: Security requirements

Area	Requirement	Verification
Authentication	Verify identity before protected operations	Login and session tests
Authorization	Enforce role and record-ownership boundaries	Cross-account access tests
Password security	Hash stored passwords	Database inspection
Input validation	Validate user, vendor and booking inputs	Invalid-input tests
Query safety	Use parameterised queries	Code review, injection tests
Secrets	Keep credentials outside source repository	Repository review
Auditability	Record who changed price, availability and booking status, and when	Update-log tests

4.5 Performance, Usability, Reliability & Interoperability

- Ordinary actions (entering requirements, viewing recommendations, comparing options, reviewing the budget) should complete within a few seconds on a standard laptop for the prototype dataset; the target is confirmed during testing.
- Filtering is performed before ranking and optimization so expensive computation runs only on feasible candidates; queries retrieve only the records needed for the current request.
- The recommendation and budget modules return a controlled message when no feasible result exists, and the application never displays success when an underlying operation has failed.
- Forms use clear labels, validation messages and input guidance; navigation is consistent across requirement entry, recommendations, comparison, budget review, recalculation, booking and vendor management.
- The application clearly shows system status: processing status, validation errors, recommendation status, recalculation results and booking confirmation status.

Table 4.10: Quality requirements

Area	Requirement
Responsiveness	Normal planning actions respond within the prototype target under expected load.
Database efficiency	Indexed columns and bounded queries for filtering.
Reliability	Invalid input, unavailable records and failed operations are handled safely without data corruption.
Usability	Clear labels, readable results, simple navigation and understandable explanations.
Accessibility	Readable typography, adequate contrast, and information that does not depend on colour alone.
Maintainability	Modular code, documented interfaces and version-controlled data and configuration.
Interoperability	Documented, consistent data structures between modules and the database.

4.6 Testing, Validation & Operational Acceptance

- Testing is planned around individual modules and the end-to-end planning workflow; each requirement has a matching validation method wherever practical.

- **Extraction testing:** form and natural-language inputs produce correct structured parameters; ambiguous or incomplete inputs lead to a clarification request.
- **Recommendation testing:** hard filtering, preference scoring, ranking order and explanation output are verified against manually prepared scenarios with known expected results. If a supervised model is used, suitable ranking or classification metrics (precision, recall, F1-score or ranking metrics) are reported together with the dataset and split used.
- **Budget testing:** component calculations, budget limits, feasible combinations, alternatives and over-budget cases, including boundary cases (total exactly equal to budget, zero optional services, very small budget, guest count at venue capacity limit).
- **Recalculation testing:** each change type in Table 3.9 produces an updated plan without a manual restart, and the change summary agrees with the actual differences between the two plans.
- **Vendor dashboard testing:** price, package, service and availability updates are saved and used by the next customer request.
- **Booking testing:** successful confirmation, unavailable dates, invalid selections, concurrent attempts on the same date, vendor price change after confirmation, and retrieval of the stored confirmation.
- **Operational acceptance:** the database can be created from scripts, the application starts from documented instructions, critical test scenarios pass, and test results are recorded in the final report.
- **Planned evaluation:** functional test cases, recommendation-relevance checks against manually curated scenarios, cost-calculation test cases, and usability feedback. Where a supervised model is used, metrics such as precision, recall, F1-score, or ranking metrics such as Precision@k are reported according to the model selected.
- **Planned test coverage:** requirement extraction accuracy, recommendation and comparison output, budget calculation correctness, event plan recalculation, vendor-side update propagation, booking confirmation and price protection, database operations, and overall system performance.

Table 4.11: Representative test scenarios

ID	Scenario	Expected Result
T1	Natural-language input with event type, guests, budget and location	All four fields extracted correctly
T2	Input missing the guest count	System asks for the guest count
T3	Guest count exceeds every venue capacity	No options; message identifies capacity as the cause
T4	Vendor marks the requested date unavailable	Venue is excluded from recommendations
T5	Plan total exceeds the budget	Alternative plans are offered with cost savings
T6	Guest count increased after plan generation	Capacity re-checked; costs and options updated; change summary shown
T7	Vendor changes price after booking confirmation	Confirmed booking keeps the agreed price
T8	Two customers try to confirm the same date	Only one booking succeeds
T9	Customer attempts to edit another customer's booking	Request rejected

Table 4.12: Verification summary

Requirement	Implementation Area	Verification
Requirement extraction	NLP / input module	Input-output and scenario testing

Requirement	Implementation Area	Verification
Recommendation	Filtering and ranking	Curated relevance scenarios
Budget optimization	Cost and optimization logic	Calculation and boundary tests
Plan recalculation	Recalculation module	Requirement-change test cases
Vendor updates	Vendor dashboard and database	Update-propagation tests
Booking protection	Booking module and database	Post-confirmation price-change test
Database integrity	MySQL data layer	CRUD and relationship tests
Overall workflow	Integrated application	End-to-end testing

4.7 Requirement Baseline & Traceability

- The baseline defines the platform as an integrated decision-support and event planning system rather than a simple banquet listing website, covering requirement understanding, vendor and venue data management, recommendation, comparison, budget optimization, recalculation, booking protection, database services, security, usability and testing.
- Traceability connects each major requirement to its module, implementation area and verification activity. Functional completion requires the corresponding data, application logic, interface and testing components to operate together.
- Any later change is assessed for its effect on recommendation logic, database schema, vendor workflow, booking behaviour, user interface and testing, and recorded with date and reason. Code, datasets and configuration remain under version control.
- The approved structure is the direct baseline for System Design, Database Design, UI/UX Design, Implementation, Testing and the final Major Project documentation.
- **Distinguishing requirements:** preference-aware recommendation with explanations, budget optimization, automatic event plan recalculation, vendor-managed current prices and availability, and protection of agreed booking details after confirmation.

Table 4.13: Requirement traceability matrix

Requirement	Module	Implementation Area	Verification
Customer requirement gathering	Customer / NLP module	Form input and NLP processing	Input-output testing
Recommendation	Recommendation module	Filtering and ranking	Scenario testing
Explainability	Recommendation module	Score factors and explanation output	Explanation consistency check
Comparison	Comparison module and UI	Side-by-side attribute display	Parameter consistency testing
Budget optimization	Budget module	Cost and optimization logic	Calculation testing
Plan recalculation	Recalculation module	Change detection and re-execution	Requirement-change tests
Vendor availability and pricing	Vendor dashboard and database	Availability and pricing tables	Update and availability testing
Booking protection	Booking module and database	Confirmed booking record with price snapshot	Post-confirmation price-change test

Requirement	Module	Implementation Area	Verification
Security	Authentication and access layer	Role checks, hashing, validation	Access-control and validation tests
Usability	Streamlit interface	Forms, navigation, status messages	Workflow testing

4.8 Future Scope (Excluded from Prototype)

The following are excluded from the prototype and may be considered only as FUTURE SCOPE:

Table 4.14: Future scope (not part of the prototype commitment)

Item	Note
Online payment processing	Booking currently ends at confirmation record generation.
Map-based location features	Map-based location and travel-time consideration for vendor and venue selection; location is matched against a controlled list in the prototype.
Dynamic or demand-based pricing	Dynamic pricing informed by historical booking data; prices come from vendor-maintained data in the prototype.
Advanced hybrid recommendation models	To be considered as sufficient interaction data becomes available.
Multilingual input	Including Hindi/Hinglish input support; NLP is limited to the language supported by the prototype.
Calendar and reminders	Calendar integration and automated reminders for confirmed events; not required for the prototype.
Cloud deployment	Cloud deployment and a scalable multi-user architecture; the prototype runs on a local environment.

5. Summary Matrix of Decisions

Parameter	Selected Logic / Rule
Primary Region	Supported cities/areas defined by the dataset (Currency: single currency per profile, INR in examples)
Platform / Interface	Streamlit application with separate customer and vendor views
Authentication	Email + Password (salted hash), role-based access
Vendor Onboarding	Business name and service category mandatory
AI Features	Natural-language requirement extraction, preference-aware ranking, explainability, budget optimization, plan recalculation
Pricing Models	Fixed price or per-guest price for venues and services; package price replaces included service prices
Vendor Data Management	Direct database updates by vendors (not a machine-learning function); offline-booked dates marked unavailable and excluded from customer availability
Recommendation Logic	Hard-constraint filtering (capacity, location, date, event type, budget feasibility) followed by attribute-based weighted preference scoring (Algorithm A) with deterministic tie-breaking
Budget Handling	Budget is a hard constraint; best preference-score combination within budget; alternatives offered when over budget

Parameter	Selected Logic / Rule
Recalculation	Automatic re-run of affected filtering, ranking and budget logic on unconfirmed plans; previous plan version preserved
Booking Mode	Customer-confirmed booking with re-validation of price and availability
Booking Protection	Agreed price and details stored as a snapshot; later vendor changes never alter confirmed bookings
Cancellation Policy	Cancellation and modification not required in prototype
Payment Gateway / Payouts	Not part of prototype (FUTURE SCOPE)
Dispute Authority	Disputes over confirmed bookings are resolved directly between customer and vendor (outside platform scope); administrators cannot alter confirmed booking values
Database	MySQL with primary/foreign keys, constraints, transactions and indexes on filter columns
Technology Stack	Python, Pandas, NumPy, Scikit-learn, spaCy / rule-based NLP, Streamlit, MySQL, Matplotlib / Plotly, Git / GitHub
Data Policy	Curated sample/synthetic data clearly labelled with provenance; no real-world vendor claims